

本节讲解的例题及其囊括的知识点，如表6-4所示。

表6-4 三维几何基础例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 6-11	UVa 11275	3D Triangles	线段和三角形相交	刘汝佳
例 6-12	UVa 1469	Ardenia	线段之间的距离 (3D)	刘汝佳
例 6-13	UVa 1438	Asteroids	三维凸包：点到平面的距离	刘汝佳
例 6-14	UVa 1100	Paperweight	三维几何综合	刘汝佳
例 6-15	HDU 4244	The Golden Ceiling	三维几何：立方体切割	陈锋

6.5 几何专题算法

本节的题目涉及综合多种算法思路的一些几何相关问题的算法实现，包括仿射变换、离散化、扫描线、运动规划等。

例6-16 【二维仿射变换；枚举】仿射变换 (Affine Mess, World Finals 2011, HDU 3838)

平面上有一幅画，其中有3个特殊点，坐标分别为 (x_1, y_1) 、 (x_2, y_2) 和 (x_3, y_3) ，其中 $-500 \leq x_i, y_i \leq 500$ 。

首先，我们对画进行旋转变换。旋转时，先在中心为 $(0,0)$ 、边长为20的正方形边界上指定一个点 (x,y) ，然后把 x 轴正半轴旋转到穿过 (x,y) 。旋转后，所有顶点都会被捕捉到最近的整点（如果小数部分恰好为0.5，捕捉到离0较远的坐标点）。

接下来，是放大和平移（顺序未知）。放大的中心总是 $(0,0)$ ，放大比例为非0整数（可以是负数， x 方向和 y 方向的放大比例不一定相同）； x 和 y 的平移量均为整数。

现在，只知道画上的3个特殊点在变换后的坐标为 (x_4, y_4) 、 (x_5, y_5) 和 (x_6, y_6) ，但不知道哪个点对应哪个点。问：可以确定变换方式吗？如果有多种可能的变换，它们是等价的吗？

【代码实现】

```
// 刘汝佳
#include <cstdio>
#include <cmath>
#include <vector>
#include <algorithm>
using namespace std;
```

```
/*
```

求解下列方程组的整数解的个数：

$$p*s + d = x \quad (1)$$

$$q*s + d = y \quad (2)$$

$$r*s + d = z \quad (3)$$

其中， s 代表缩放系数， d 代表平移量

解：联立方程(1),(2)，得 $(p-q)*s = x-y$

i) 如果 $p-q = 0$ ，则必须有 $x == y$ ，否则无解

ii) 如果 $p-q \neq 0$ ，则 $s = (x-y)/(p-q)$ 。如果 s 不是整数，则无解，否则 s 是一个解

类似地，还应联立方程(2),(3)和方程(3),(1)求解。

i) 如果求出了多个 s ，它们必须相同

ii) 如果一个 s 都没有得到，说明有无穷多组解（返回 2 就可以了）

iii) 如果 $s = 0$ ，根据题意，也无解

```
*/
int solve(int p, int q, int r, int x, int y, int z) {
    int a[] = {p, q, r};
    int b[] = {x, y, z};
    vector<int> ans;
    for(int i = 0; i < 3; i++) {
        int P = a[i], Q = a[(i+1)%3], X = b[i], Y = b[(i+1)%3];
        if(P == Q) { if(X != Y) return 0; }
        else if((X - Y) % (P - Q) != 0) return 0;
        else ans.push_back((X - Y) / (P - Q));
    }
    if(ans.empty()) return 2; // 3 个方程等价，无穷多组解
    sort(ans.begin(), ans.end());
    if(ans[0] != ans.back() || ans[0] == 0) return 0; // 求出的 s 不全相同或者等于 0
    return 1;
}

int x[3], y[3]; // 变换前的点
int x2[3], y2[3]; // 变换后的点
int ix[3], iy[3]; // 旋转+捕捉 E 后的点

int main() {
    int kase = 0;
    for(;;) {
        int ok = 0;
        for(int i = 0; i < 3; i++) {
            scanf("%d%d", &x[i], &y[i]);
            if(x[i] != 0 || y[i] != 0) ok = 1;
        }
        if(!ok) break;
        for(int i = 0; i < 3; i++) scanf("%d%d", &x2[i], &y2[i]);
        int ans = 0; // 解的个数

        // 枚举旋转方式
        // 注意，旋转 180 度等价于缩放 (-1, -1)，所以只枚举 40 个点而不是 80 个
        for(int i = 0; i < 40; i++) {
            int rx, ry;
            if(i < 20) { rx = 10; ry = i - 10; } // (10, -10), (10, -9), ..., (10, 9), (10, 9)
            else { rx = 30 - i; ry = 10; } // (10, 10), (9, 10), ..., (-9, 10)

            // 变换前 3 个点，保存在 (ix[i], iy[i]) 中
            double len = sqrt(rx*rx+ry*ry);
            double cosa = rx / len;
            double sina = ry / len;
```

```

int ix[3], iy[3];
for(int j = 0; j < 3; j++) {
    ix[j] = (int)floor(x[j] * cosa - y[j] * sina + 0.5);
    iy[j] = (int)floor(x[j] * sina + y[j] * cosa + 0.5);
}

// 枚举(ix, iy)和(x2, y2)的对应关系
int p[3] = {0, 1, 2};
do {
    int cnt1 = solve(ix[0], ix[1], ix[2], x2[p[0]], x2[p[1]], x2[p[2]]);
    int cnt2 = solve(iy[0], iy[1], iy[2], y2[p[0]], y2[p[1]], y2[p[2]]);
    ans += cnt1 * cnt2;           // x, y 方向独立, 分别求解
} while(next_permutation(p, p+3));
}

printf("Case %d: ", ++kase);
if(ans == 0) printf("no solution\n");
else if(ans == 1) printf("equivalent solutions\n");
else printf("inconsistent solutions\n");
}
return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 6.4.1 节例题 12
*/

```

例6-17 【三维仿射变换；平面的变换】组合变换（Composite Transformations, UVa 12303）

空间中有 n 个点和 m 个平面，你的任务是按顺序向它们施加 t ($1 \leq n, m \leq 50\,000$, $1 \leq t \leq 1000$) 个变换，输出每个点的最终位置和每个平面的最终方程。一共有3种变换，如表6-5所示。

表6-5 3种变换操作

变 换	说 明
TRANSLATE $a\ b\ c$	点 (x,y,z) 变成 $(x+a,y+b,z+c)$ (a, b, c 不全为 0)
ROTATE $a\ b\ c\ \theta$	把每个点旋转 θ 度。旋转轴是向量 (a,b,c) ，并且穿过原点。当旋转轴指向你时，旋转看上去是逆时针的。参数 θ 是 $0 \sim 359$ 的整数
SCALE $a\ b\ c$	点 (x,y,z) 变成 (ax,by,cz)

对于每个点，输出一行，即变换后的坐标。对于每个平面，输出一行，即变换后平面方程的参数 a , b , c 和 d 。输出应保证

$a^2+b^2+c^2=1$ ，如果有多种表示方法，输出任意一种均可。输出应保留两位小数。本题允许有一定的浮点误差。

【代码实现】

```
// 刘汝佳
#include <cstdio>
#include <cmath>
#include <cstdlib>
```

```

#include <cstring>
#include <cassert>
using namespace std;

const double PI = acos(-1.0);

struct Point3 {
    double x, y, z;
    Point3(double x=0, double y=0, double z=0):x(x),y(y),z(z) { }
};

typedef Point3 Vector3;
Vector3 operator + (const Vector3& A, const Vector3& B)
{ return Vector3(A.x+B.x, A.y+B.y, A.z+B.z); }
Vector3 operator - (const Point3& A, const Point3& B)
{ return Vector3(A.x-B.x, A.y-B.y, A.z-B.z); }
Vector3 operator * (const Vector3& A, double p)
{ return Vector3(A.x*p, A.y*p, A.z*p); }
Vector3 operator / (const Vector3& A, double p)
{ return Vector3(A.x/p, A.y/p, A.z/p); }

double Dot(const Vector3& A, const Vector3& B)
{ return A.x*B.x + A.y*B.y + A.z*B.z; }
double Length(const Vector3& A) { return sqrt(Dot(A, A)); }
Vector3 Cross(const Vector3& A, const Vector3& B)
{ return Vector3(A.y*B.z - A.z*B.y, A.z*B.x - A.x*B.z, A.x*B.y - A.y*B.x); }

// 平面
struct Plane {
    double a, b, c, d;
    Plane() {}
    Plane(Point3* P) { // 用三点确定一个平面，调用者须保证三点不共线
        Vector3 V = Cross(P[1]-P[0], P[2]-P[0]);
        V = V / Length(V);
        a = V.x; b = V.y; c = V.z; d = -Dot(V, P[0]);
    }
    Point3 sample() const { // 随机采样
        double v1 = rand() / (double)RAND_MAX;
        double v2 = rand() / (double)RAND_MAX;
        if(a != 0) return Point3(-(d+v1*b+v2*c)/a, v1, v2);
        if(b != 0) return Point3(v1, -(d+v1*a+v2*c)/b, v2);
        if(c != 0) return Point3(v1, v2, -(d+v1*a+v2*b)/c);
        assert(0); // 不是一个平面
    }
};

// 4x4 齐次变换矩阵
struct Matrix4x4 {
    double v[4][4];

```

```

// 矩阵乘法
inline Matrix4x4 operator * (const Matrix4x4 &rhs) const {
    Matrix4x4 ans;
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++) {
            ans.v[i][j] = 0;
            for (int k = 0; k < 4; k++)
                ans.v[i][j] += v[i][k] * rhs.v[k][j];
        }
    return ans;
}

// 变换一个点, 相当于右乘列向量(x, y, z, 1)
inline Point3 transform(Point3 P) const {
    double p[4] = {P.x, P.y, P.z, 1}, ans[4] = {0};
    for(int i = 0; i < 4; i++)
        for(int k = 0; k < 4; k++)
            ans[i] += v[i][k] * p[k];
    return Point3(ans[0], ans[1], ans[2]); // ans[3]肯定是1
}

// 单位矩阵
void loadIdentity() {
    memset(v, 0, sizeof(v));
    v[0][0] = v[1][1] = v[2][2] = v[3][3] = 1;
}

// 平移矩阵
void loadTranslate(double a, double b, double c) {
    loadIdentity();
    v[0][3] = a; v[1][3] = b; v[2][3] = c;
}

// 缩放矩阵
void loadScale(double a, double b, double c) {
    loadIdentity();
    v[0][0] = a; v[1][1] = b; v[2][2] = c;
}

// 绕固定轴旋转一定角度的矩阵
void loadRotation(double a, double b, double c, double deg) {
    loadIdentity();
    double rad = deg / 180 * PI;
    double sine = sin(rad), cosine = cos(rad);
    Vector3 L(a, b, c);
    L = L / Length(L);
    v[0][0] = cosine + L.x * L.x * (1.0 - cosine);
    v[0][1] = L.x * L.y * (1 - cosine) - L.z * sine;

```

```

    v[0][2] = L.x * L.z * (1 - cosine) + L.y * sine;
    v[1][0] = L.y * L.x * (1 - cosine) + L.z * sine;
    v[1][1] = cosine + L.y * L.y * (1 - cosine);
    v[1][2] = L.y * L.z * (1 - cosine) - L.x * sine;
    v[2][0] = L.z * L.x * (1 - cosine) - L.y * sine;
    v[2][1] = L.z * L.y * (1 - cosine) + L.x * sine;
    v[2][2] = cosine + L.z * L.z * (1 - cosine);
}
};

const int maxn = 50000 + 10;
const int maxp = 50000 + 10;
Point3 P[maxn];
Plane planes[maxp];

int main() {
    int n, m, T;
    scanf("%d%d%d", &n, &m, &T);
    for(int i = 0; i < n; i++)
        scanf("%lf%lf%lf", &P[i].x, &P[i].y, &P[i].z);
    for(int i = 0; i < m; i++)
        scanf("%lf%lf%lf%lf", &planes[i].a, &planes[i].b, &planes[i].c, &planes[i].d);

    // 点P将被变换为 M[T-1] * ... * M[2] * M[1] * M[0] * P
    // 根据结合律, 先计算mat = (M[T-1] * ... * M[0]), 则点P变换为mat * P
    Matrix4x4 mat;
    mat.loadIdentity();
    for(int i = 0; i < T; i++) {
        char op[100];
        double a, b, c, theta;
        scanf("%s%lf%lf%lf", op, &a, &b, &c);
        Matrix4x4 M;
        if(op[0] == 'T') M.loadTranslate(a, b, c);
        else if(op[0] == 'S') M.loadScale(a, b, c);
        else if(op[0] == 'R') {
            scanf("%lf", &theta); M.loadRotation(a, b, c, theta);
        }
        mat = M * mat;
    }

    // 变换点
    for(int i = 0; i < n; i++) {
        Point3 ans = mat.transform(P[i]);
        printf("%.2lf %.2lf %.2lf\n", ans.x, ans.y, ans.z);
    }
    // 变换平面
    for(int i = 0; i < m; i++) {
        Point3 A[3];
        for(int j = 0; j < 3; j++) A[j] = mat.transform(planes[i].sample());
        Plane pl(A);
    }
}

```



```

    printf("%.2lf %.2lf %.2lf %.2lf\n", pl.a, pl.b, pl.c, pl.d);
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 6.4.1 节例题 13
*/

```

例6-18 【直线型的离散化】山的轮廓线 (The Sky is the Limit, World Finals 2008, UVa 1077)

输入 n ($1 \leq n \leq 100$) 座山的信息，计算它们的轮廓线长度（即从天空往下看，可以看到的线段的总长度）。如图6-10所示，虚线是不可见的，山和山之间的空白地带不算在轮廓线内，每座山都是一个等腰三角形。



图6-10 山的轮廓线

【代码实现】

```

// 刘汝佳
#include <cstdio>
#include <cmath>
#include <algorithm>
using namespace std;

const double eps = 1e-10;
int dcmp(double x) {
    if(fabs(x) < eps) return 0;
    return x < 0 ? -1 : 1;
}
struct Point {
    double x, y;
    Point(double x=0, double y=0):x(x),y(y) { }
};
typedef Point Vector;
Vector operator + (const Vector& A, const Vector& B)
{ return Vector(A.x+B.x, A.y+B.y); }
Vector operator - (const Point& A, const Point& B)
{ return Vector(A.x-B.x, A.y-B.y); }
Vector operator * (const Vector& A, double p)
{ return Vector(A.x*p, A.y*p); }
bool operator < (const Point& a, const Point& b)
{ return a.x < b.x || (a.x == b.x && a.y < b.y); }
bool operator == (const Point& a, const Point &b)
{ return dcmp(a.x-b.x) == 0 && dcmp(a.y-b.y) == 0; }
double Dot(const Vector& A, const Vector& B) { return A.x*B.x + A.y*B.y; }

```

```

double Cross(const Vector& A, const Vector& B) { return A.x*B.y - A.y*B.x; }
double Length(const Vector& A) { return sqrt(Dot(A, A)); }

Point GetLineIntersection(const Point& P, const Vector& v,
    const Point& Q, const Vector& w) {
    Vector u = P-Q;
    double t = Cross(w, u) / Cross(v, w);
    return P+v*t;
}

bool SegmentProperIntersection(const Point& a1, const Point& a2,
    const Point& b1, const Point& b2) {
    double c1 = Cross(a2-a1, b1-a1), c2 = Cross(a2-a1, b2-a1),
    c3 = Cross(b2-b1, a1-b1), c4 = Cross(b2-b1, a2-b1);
    return dcmp(c1)*dcmp(c2)<0 && dcmp(c3)*dcmp(c4)<0;
}

const int maxn = 100 + 10;
Point P[maxn], L[maxn][2][2];
double x[maxn*maxn];

int main() {
    int n, kase = 0;
    while(scanf("%d", &n) == 1 && n) {
        int c = 0;
        for(int i = 0; i < n; i++) {
            double X, H, B;
            scanf("%lf%lf%lf", &X, &H, &B);
            L[i][0][0] = Point(X-B*0.5, 0);
            L[i][0][1] = L[i][1][0] = Point(X, H);
            L[i][1][1] = Point(X+B*0.5, 0);
            x[c++] = X-B*0.5;
            x[c++] = X;
            x[c++] = X+B*0.5;
        }
        for(int i = 0; i < n; i++) for(int a = 0; a < 2; a++)
            for(int j = i+1; j < n; j++) for(int b = 0; b < 2; b++) {
                Point P1 = L[i][a][0], P2 = L[i][a][1], P3 = L[j][b][0], P4 = L[j][b][1];
                if(SegmentProperIntersection(P1, P2, P3, P4))
                    x[c++] = GetLineIntersection(P1, P2-P1, P3, P4-P3).x;
            }

        // 根据所有交点离散化
        sort(x, x+c);
        c = unique(x, x+c) - x;

        double ans = 0;
        Point lastp;
        for(int k = 0; k < c; k++) {
            // 计算直线 x=x[k] 和山相交的最高点

```

```

    Point P(x[k], 0);
    Vector V(0, 1);
    double maxy = -1;
    for(int i = 0; i < n; i++) for(int a = 0; a < 2; a++) {
        Point P1 = L[i][a][0], P2 = L[i][a][1];
        Point intersection = GetLineIntersection(P, V, P1, P2-P1);
        if(dcmp(intersection.x-P1.x) >= 0 && dcmp(intersection.x-P2.x) <= 0)
            maxy = max(maxy, intersection.y);
    }
    Point newp(x[k], maxy);
    if(k > 0 && (dcmp(lastp.y) > 0 || dcmp(maxy) > 0)) ans += Length(newp - lastp);
    lastp = newp;
}

printf("Case %d: %.0lf\n\n", ++kase, ans);
}
return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 6.4.2 节例题 14
*/

```

例6-19 【圆的离散化】核电厂 (Nuclear Plants, UVa 1367)

假设某国的领土是一个长 n 千米、宽 m 千米的矩形 ($1 \leq n, m \leq 10\,000$)，边平行于坐标轴，左上角的坐标为 $(0,0)$ ，右下角的坐标为 (n,m) 。该国领土上有一些核电厂。核电厂有大小之分，数目不超过100，小核电厂周围0.58千米内都不能种植大麦，大核电厂周围1.31千米内都不能种植大麦。你的任务是计算出可以种植大麦的区域的总面积。

【代码实现】

```
// 刘汝佳
#include <cstdio>
#include <cmath>
#include <cstring>
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

const double eps = 5 * 1e-13;
int dcmp(double x) {
    if (fabs(x) < eps) return 0; else return x < 0 ? -1 : 1;
}

const double PI = acos(-1);
const double TWO_PI = PI * 2;

double NormalizeAngle(double rad, double center = PI) {
    return rad - TWO_PI * floor((rad + PI - center) / TWO_PI);
}
```

```

}

struct Point {
    double x, y;
    Point(double x = 0, double y = 0): x(x), y(y) { }
};

typedef Point Vector;

Vector operator + (Vector A, Vector B) { return Vector(A.x + B.x, A.y + B.y); }
Vector operator - (Point A, Point B) { return Vector(A.x - B.x, A.y - B.y); }
Vector operator * (Vector A, double p) { return Vector(A.x * p, A.y * p); }
Vector operator / (Vector A, double p) { return Vector(A.x / p, A.y / p); }

/* 理论上这个“小于”运算符是错误的，因为可能有3个点a, b, c, a和b很接近（即a<b和b<a都不成立），b和c很接近，但a和c不接近。所以使用这种“小于”运算符的前提是能排除上述情况 */
bool operator < (const Point& a, const Point& b) {
    return dcmp(a.x - b.x) < 0 || (dcmp(a.x - b.x) == 0
        && dcmp(a.y - b.y) < 0);
}

bool operator == (Point A, Point B)
{ return dcmp(A.x - B.x) == 0 && dcmp(A.y - B.y) == 0; }
double Dot(Vector A, Vector B) { return A.x * B.x + A.y * B.y; }
double Length(Vector A) { return sqrt(Dot(A, A)); }
double Cross(Vector A, Vector B) { return A.x * B.y - A.y * B.x; }
double angle(Vector v) { return atan2(v.y, v.x); }
bool OnSegment(const Point& p, const Point& a1, const Point& a2) {
    return dcmp(Cross(a1 - p, a2 - p)) == 0
        && dcmp(Dot(a1 - p, a2 - p)) < 0;
}

// 交点相对于圆1的极角保存在rad中
void getCircleCircleIntersection(Point c1, double r1,
    Point c2, double r2, vector<double>& rad) {
    double d = Length(c1 - c2);
    if (dcmp(d) == 0) return; // 不管是内含还是重合，都不相交
    if (dcmp(r1 + r2 - d) < 0) return;
    if (dcmp(fabs(r1 - r2) - d) > 0) return;

    double a = angle(c2 - c1);
    double da = acos((r1 * r1 + d * d - r2 * r2) / (2 * r1 * d));
    rad.push_back(NormalizeAngle(a - da));
    rad.push_back(NormalizeAngle(a + da));
}

Point GetLineProjection(Point P, Point A, Point B) {
    Vector v = B - A;

```

```

    return A + v * (Dot(v, P - A) / Dot(v, v));
}

// 直线AB和圆心为C、半径为r的圆的交点相对于圆的极角保存在rad中
void getLineCircleIntersection(Point A, Point B, Point C, double r,
vector<double>& rad) {
    Point p = GetLineProjection(C, A, B);
    double a = angle(p - C);
    double d = Length(p - C);
    if (dcmp(d - r) > 0) return;
    if (dcmp(d) == 0) { // 过圆心
        rad.push_back(NormalizeAngle(angle(A - B)));
        rad.push_back(NormalizeAngle(angle(B - A)));
    }
    double da = acos(d / r);
}

////////// 题目相关
const int maxn = 200 + 5;
int n, N, M; // n是圆的总数, N和M是场地长宽
Point P[maxn];
double R[maxn];

// 取圆no中弧度为rad的点
Point getPoint(int no, double rad) {
    return Point(P[no].x + cos(rad) * R[no], P[no].y + sin(rad) * R[no]);
}

// 第no个圆中弧度为rad的点是否可见。相同的圆只有编号最小的可见（虽然对于本题来说不必要）
bool visible(int no, double rad) {
    Point p = getPoint(no, rad);
    if (p.x < 0 || p.y < 0 || p.x > N || p.y > M) return false;
    for (int i = 0; i < n; i++) {
        if (P[no] == P[i] && dcmp(R[no] - R[i]) == 0 && i < no) return false;
        if (dcmp(Length(p - P[i]) - R[i]) < 0) return false;
    }
    return true;
}

// 场地边界上的点p是否可见
bool visible(Point p) {
    for (int i = 0; i < n; i++) {
        if (dcmp(Length(p - P[i]) - R[i]) <= 0) return false;
    }
    return true;
}

/* 求圆的并在(0,0)-(N,M)内的面积
使用一般曲边图形的面积算法。下文中,“所求图形”指的是不能种植大麦的区域,

```

它的边界由圆弧和直线段构成。

算法：对于所求图形边界上的每一段（可以是曲线） $a \rightarrow b$ ，

累加 $Cross(a, b)$ 和它在直线段 $a \rightarrow b$ 右边部分的面积（左边部分算负）

边界计算：

1. 每个圆被其他圆和场地边界分成了若干条圆弧，中点不被其他圆覆盖，且场地内的圆弧在所求图形边界上

2. 场地的 4 条边界被圆分成了若干条线段。中点在某个圆内部的线段在所求图形边界上 */

```
double getArea() {
    Point b[4];
    b[0] = Point(0, 0);
    b[1] = Point(N, 0);
    b[2] = Point(N, M);
    b[3] = Point(0, M);
    double area = 0;

    // 圆弧部分
    for (int i = 0; i < n; i++) {
        vector<double> rad;
        rad.push_back(0);
        rad.push_back(Pi * 2);

        // 圆和边界的交点
        for (int j = 0; j < 4; j++)
            getLineCircleIntersection(b[j], b[(j + 1) % 4], P[i], R[i], rad);

        // 圆和圆的交点
        for (int j = 0; j < n; j++)
            getCircleCircleIntersection(P[i], R[i], P[j], R[j], rad);

        sort(rad.begin(), rad.end());
        for (int j = 0; j < rad.size() - 1; j++) if (rad[j + 1] - rad[j] > eps) {
            double mid = (rad[j] + rad[j + 1]) / 2.0; // 圆弧中点相对于圆 i 圆心的极角
            if (visible(i, mid)) { // 弧中点可见，因此弧在图形边界上
                area += Cross(getPoint(i, rad[j]), getPoint(i, rad[j + 1])) / 2.0;
                double a = rad[j + 1] - rad[j];
                area += R[i] * R[i] * (a - sin(a)) / 2.0;
            }
        }
    }

    // 直线段部分
    for (int i = 0; i < 4; i++) {
        Vector v = b[(i + 1) % 4] - b[i];
        double len = Length(v);

        vector<double> dist;
        dist.push_back(0);
        dist.push_back(len);
        for (int j = 0; j < n; j++) {
```



```

    vector<double> rad;
    getLineCircleIntersection(b[i], b[(i + 1) % 4], P[j], R[j], rad);
    for (int k = 0; k < rad.size(); k++) {
        Point p = getPoint(j, rad[k]);
        dist.push_back(Length(p - b[i]));
    }
}
// 必须按照到起点的距离排序，而不能按照点的字典序排序，否则向量方向可能会反
sort(dist.begin(), dist.end());
vector<Point> points;
for (int j = 0; j < dist.size(); j++)
    points.push_back(b[i] + v * (dist[j] / len));

for (int j = 0; j < dist.size() - 1; j++) {
    Point midp = (points[j] + points[j + 1]) / 2.0;
    if (!visible(midp)) // 线段中点不可见，因此线段在图形边界上
        area += Cross(points[j], points[j + 1]) / 2.0;
}
}

return N * M - area;
}

int main() {
    int ks, kl;
    while (scanf("%d%d%d%d", &N, &M, &ks, &kl) == 4 && N && M) {
        for (int i = 0; i < ks; i++)
            scanf("%lf%lf", &P[i].x, &P[i].y), R[i] = 0.58;
        sort(P, P + ks);
        ks = unique(P, P + ks) - P;
        for (int i = 0; i < kl; i++)
            scanf("%lf%lf", &P[ks + i].x, &P[ks + i].y), R[ks + i] = 1.31;
        sort(P + ks, P + ks + kl);
        n = unique(P + ks, P + ks + kl) - P;
        printf("%.2lf\n", getArea());
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 6.4.2 节例题 15
*/

```

例6-20 【投影；离散化】可见的屋顶（Raising the Roof, World Finals 2007, UVa 1065）

给定 T ($1 \leq T \leq 1000$) 个三角形屋顶，你的任务是计算出暴露在阳光下的部分（即从天空向地面俯视时能看到的部分）的总面积。注意，被遮挡的部分不应该被算到总面积中。 xy 平面是地面， z 轴竖直向上。

输出可见部分的总面积，保留两位小数。

【代码实现】

```

// 刘汝佳
// 寻找 top 时, 改用简单循环寻找, 效率稍低但代码简单
#include <cmath>
#include <cstdio>
#define REP(i, n) for (int i = 0; i < (n); ++i)

const double eps = 1e-8;
int dcmp(double x) {
    if (fabs(x) < eps) return 0;
    return x < 0 ? -1 : 1;
}
struct Point3 {
    int x, y, z;
    Point3(int x = 0, int y = 0, int z = 0) : x(x), y(y), z(z) {}
};
typedef Point3 Vector3;
Vector3 operator-(const Point3& A, const Point3& B)
{ return Vector3(A.x - B.x, A.y - B.y, A.z - B.z); }
int Dot(const Vector3& A, const Vector3& B)
{ return A.x * B.x + A.y * B.y + A.z * B.z; }
double Length(const Vector3& A) { return sqrt(Dot(A, A)); }
Vector3 Cross(const Vector3& A, const Vector3& B) {
    return Vector3(A.y * B.z - A.z * B.y, A.z * B.x - A.x * B.z,
        A.x * B.y - A.y * B.x);
}

#include <algorithm>
#include <cstdlib>
#include <cstring>
#include <vector>
using namespace std;

const int maxn = 300 + 10, maxt = 1000 + 10;
Point3 p[maxn];
int n, m;
int t[maxt][3];
Vector3 normal[maxt]; // 三角形 i 的法向量
double d[maxt]; // 三角形 i 的点法式为 Dot(normal[i], p) = d
double area_ratio[maxt]; // 三角形 i 的投影面积乘以 area_ratio[i] 就是实际面积

// 输入中有在竖直平面内 (即 normal[i].z=0) 的三角形,
// 但主算法会自动忽略它们, 不用担心 area_ratio[i] 不存在
void init() {
    for (int i = 0; i < m; i++) {
        Point3 p0 = p[t[i][0]], p1 = p[t[i][1]], p2 = p[t[i][2]];
        normal[i] = Cross(p1 - p0, p2 - p0);
        d[i] = Dot(normal[i], p0);
        if (normal[i].z != 0)

```

```

        area_ratio[i] = fabs((double)Length(normal[i]) / normal[i].z);
    }
}

inline double getTriangleZ(int idx, double x, double y) {
    return (d[idx] - normal[idx].x * x - normal[idx].y * y) / normal[idx].z;
}

struct Event {
    int id; // 涉及的三角形编号
    double y; // 与扫描线交点的 y 坐标
    Event(int id, double y) : id(id), y(y) {}
    bool operator<(const Event& rhs) const { return y < rhs.y; }
};

double solve() {
    // 离散化
    vector<double> sx;
    for (int i = 1; i <= n; i++) sx.push_back(p[i].x);
    REP(i, m) REP(j, m) REP(a, 3) REP(b, 3) {
        // 求 pa-pb 和 qa-qb 投影到 xy 平面后的交点。直接解参数方程
        Point3 pa = p[t[i][a]];
        Point3 pb = p[t[i][(a + 1) % 3]];
        Point3 qa = p[t[j][b]];
        Point3 qb = p[t[j][(b + 1) % 3]];
        int dpx = pb.x - pa.x;
        int dpy = pb.y - pa.y;
        int dqx = qb.x - qa.x;
        int dqy = qb.y - qa.y;
        int deno = dpx * dqy - dpy * dqx;
        if (deno == 0) continue;
        double t = (double)(dqy * (qa.x - pa.x) + dqx * (pa.y - qa.y)) / deno;
        double s = (double)(dpy * (qa.x - pa.x) + dpx * (pa.y - qa.y)) / deno;
        if (t > 1 || t < 0 || s > 1 || s < 0) continue;
        sx.push_back(pa.x + t * dpx);
    }
    sort(sx.begin(), sx.end());
    sx.erase(unique(sx.begin(), sx.end()), sx.end());

    double ans = 0;
    for (int i = 0; i < sx.size() - 1; i++) {
        // 扫描线位于 x = xx
        double xx = (sx[i] + sx[i + 1]) / 2;
        // 计算扫描线穿过的三角形集合，为每个三角形创建“进入”和“离开”事件
        vector<Event> events;
        REP(j, m) if (normal[j].z != 0) REP(a, 3) { // 忽略竖直线内的三角形
            Point3 pa = p[t[j][a]], pb = p[t[j][(a + 1) % 3]];
            // 计算扫描线 x = xx 和 pa-pb 在平面 xy 上投影的交点
            if (pa.x == pb.x) continue; // 竖直线段

```

```

    if (!(min(pa.x, pb.x) <= sx[i] && max(pa.x, pb.x) >= sx[i + 1]))
        continue; // 不在竖直条内
    double y = pa.y + (pb.y - pa.y) * (xx - pa.x) / (pb.x - pa.x); // 解方程
    events.push_back(Event(j, y));
}
if (events.empty()) continue;

// 按照 y 递增的顺序处理事件
int inside[maxt];
fill_n(inside, maxt, 0), sort(events.begin(), events.end());
for (int j = 0; j < events.size() - 1; j++) {
    inside[events[j].id] ^= 1;
    if (fabs(events[j].y - events[j + 1].y) < eps)
        continue; // y 相同的事件要等到所有 inside 更新完毕后才能处理

    // 投影梯形的面积等于中线乘以高
    double proj_are = (sx[i + 1] - sx[i]) * (events[j + 1].y - events[j].y);

    // 在下一个事件发生之前, 哪个三角形在最上面
    int top = -1; // 测试 y 坐标中点, 计算 zz 误差比较小
    double topz = -1e9, yy = (events[j].y + events[j + 1].y) / 2;
    for (int k = 0; k < m; k++)
        if (inside[k]) {
            double zz = getTriangleZ(k, xx, yy);
            if (zz > topz) topz = zz, top = k; // 更新最上面的三角形编号 top
        }

    // 投影部分面积乘以比例系数等于实际面积
    if (top >= 0) ans += area_ratio[top] * proj_are;
}
}
return ans;
}

int main() {
    int kase = 0;
    while (scanf("%d%d", &n, &m) == 2 && n > 0) {
        for (int i = 1; i <= n; i++)
            scanf("%d%d%d", &p[i].x, &p[i].y, &p[i].z); // 顶点编号为 1~n
        for (int i = 0; i < m; i++) scanf("%d%d%d", &t[i][0], &t[i][1], &t[i][2]);
        init();
        double ans = solve();
        printf("Case %d: %.2lf\n\n", ++kase, ans);
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 6.4.2 节例题 16
*/

```

例6-21 【扫描法；用BST维护扫描线】画家（Painter, World Finals 2008, UVa 1075）

画家Peer在一个矩形的画布上画了 n ($n \leq 100\,000$) 个三角形。画完之后，他开始涂色。最外层的画布涂最浅的颜色，然后次外层的用次浅的颜色，以此类推。如图6-11所示，一共有4个层次的三角形，加上画布本身一共需要5种颜色。你的任务是帮助Peer在涂色之前判断出需要几种颜色。如果三角形出现相交（即其中一个三角形的某条边和另一个三角形的某条边有公共点），则涂色无法进行。

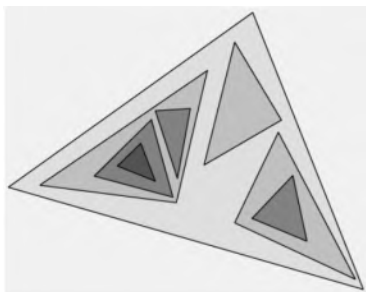


图6-11 三角形涂色

输出需要的颜色数。如果无法涂色，输出ERROR。

【代码实现】

```

// 刘汝佳
#include <cstdio>
#include <cstdlib>
#include <map>
#include <algorithm>
using namespace std;

typedef long long LL;

struct Point {
    int x, y;
    Point(int x = 0, int y = 0):x(x),y(y){}
    void read() { scanf("%d%d", &x, &y); }
    bool operator < (const Point& p) const {
        return x < p.x || x == p.x && y < p.y;
    }
    Point operator - (const Point& rhs) const {
        return Point(x - rhs.x, y - rhs.y);
    }
};

int icmp(LL x) {
    if(x == 0) return 0;
    return x > 0 ? 1 : -1;
}

inline LL Cross(Point p, Point p1, Point p2) {
    return (LL) (p1.x - p.x) * (LL) (p2.y - p.y) - (LL) (p1.y - p.y) * (LL) (p2.x - p.x);
}

// 由于线段相交判定执行次数较多, 这里进行了一些简单优化
inline bool SegmentIntersection(const Point& a1, const Point& a2, const Point&
b1, const Point& b2) {
    if(min(a1.x, a2.x) > max(b1.x, b2.x)) return false; // 快速排除
    if(min(a1.y, a2.y) > max(b1.y, b2.y)) return false;

```

```

    if(max(a1.x, a2.x) < min(b1.x, b2.x)) return false;
    if(max(a1.y, a2.y) < min(b1.y, b2.y)) return false;
    LL c1 = Cross(a1, a2, b1), c2 = Cross(a1, a2, b2);
    if(icmp(c1) * icmp(c2) > 0) return false;
    LL c3 = Cross(b1, b2, a1), c4 = Cross(b1, b2, a2);
    return icmp(c3) * icmp(c4) <= 0;
}

int curx;
const double eps = 1e-6;

struct Segment {
    Point p1, p2;
    int no; // 三角形编号
    double d;
    Segment(Point p1, Point p2, int no):p1(p1),p2(p2),no(no) {
        d = (p2.y - p1.y) / (p2.x + eps - p1.x);
    }
    double y() const { return p1.y + d * (curx + eps - p1.x); }
    bool operator < (const Segment& rhs) const { return y() < rhs.y(); }
};

inline bool Intersect(const Segment& a, const Segment& b) {
    if(a.no == b.no) return false;
    return SegmentIntersection(a.p1, a.p2, b.p1, b.p2);
}

bool error; // 是否已经出现相交线段
int max_depth; // 当前最大深度

const int INF = 200000;

// 本题这样做可以提高代码可读性，但不要在工程中这样使用，非常危险
#define L first
#define depth second

// 扫描线类，用一个multimap实现
struct Scanline {
    multimap<Segment, int> line;
    typedef multimap<Segment, int>::iterator Pos;
    void init() {
        line.clear();
        line.insert(make_pair(Segment(Point(-INF, -INF), Point(INF, -INF)), -1), 1));
        line.insert(make_pair(Segment(Point(-INF, INF), Point(INF, INF)), -1), 0));
    }
    inline Pos Prev(const Pos& p) const { return --Pos(p); }
    inline Pos Next(const Pos& p) const { return ++Pos(p); }
    inline Pos Insert(const Segment& s, int d = 0) {
        Pos x = line.insert(make_pair(s, d));

```



```

        if(Intersect(x->L, Prev(x)->L) || Intersect(x->L, Next(x)->L)) error = true;
        return x;
    }
    inline void Erase(const Pos& x) {
        if(Intersect(Prev(x)->L, Next(x)->L)) error = true;
        line.erase(x);
    }
} scanline;

struct Triangle {
    int no; // 编号
    Point P[3];
    Scanline::Pos p12, p13, p23;
    void read(int no) {
        this->no = no;
        for(int i = 0; i < 3; i++) scanf("%d%d", &P[i].x, &P[i].y);
        sort(P, P+3);
    }
    // 更新 x1 和 x2 的 depth. 其中, x1 是 p12 和 p13 中 y 较小的那个, x2 是另一个 (即 Next(x1)=x2)
    void updateDepth(const Scanline::Pos& x1, Scanline::Pos& x2) {
        int d = scanline.Prev(x1)->depth + 1;
        max_depth = max(max_depth, d);
        x1->depth = d;
        x2->depth = d - 1;
    }
    // 处理第 v 个结点
    void process(int v) {
        if(v == 0) {
            p12 = scanline.Insert(Segment(P[0], P[1], no));
            p13 = scanline.Insert(Segment(P[0], P[2], no));
            scanline.Next(p12) == p13 ? updateDepth(p12, p13) : updateDepth(p13, p12);
        }
        else if(v == 1) {
            p23 = scanline.Insert(Segment(P[1], P[2], no), p12->depth);
            scanline.Erase(p12);
        }
        else {
            scanline.Erase(p13);
            scanline.Erase(p23);
        }
    }
};

struct Event {
    int x, t, v; // x 坐标, 三角形编号和顶点编号
    Event() {}
    Event(int x, int t, int v):x(x),t(t),v(v) {}
    bool operator < (const Event& rhs) const {
        return x < rhs.x || x == rhs.x && v < rhs.v;
    }
};

```

```

    }
};

const int maxn = 100000 + 10;          // 最大三角形个数
Triangle tri[maxn];
Event events[maxn*3];

int main() {
    int n, kase = 0;
    while(scanf("%d",&n) == 1 && n >= 0) {
        error = false;
        max_depth = 1;
        scanline.init();
        for(int i = 0; i < n; i++) {
            tri[i].read(i);
            for(int j = 0; j < 3; j++)
                events[i*3+j] = Event(tri[i].P[j].x, i, j);
        }
        sort(events, events+n*3);
        for(int i = 0; i < n*3; i++) {
            curx = events[i].x;
            tri[events[i].t].process(events[i].v);
            if(error) break;
        }
        if(!error) printf("Case %d: %d shades\n", ++kase, max_depth);
        else printf("Case %d: ERROR\n", ++kase);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 6.4.2 节例题 17
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点，如表6-6所示。

表6-6 几何专题算法例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 6-16	HDU 3838	Affine Mess	二维仿射变换；枚举	刘汝佳
例 6-17	UVa 12303	Composite Transformations	三维仿射变换；平面的变换	刘汝佳
例 6-18	UVa 1077	The Sky is the Limit	直线型的离散化	刘汝佳
例 6-19	UVa 1367	Nuclear Plants	圆的离散化	刘汝佳
例 6-20	UVa 1065	Raising the Roof	投影；离散化	刘汝佳
例 6-21	UVa 1075	Painter	扫描法；用 BST 维护扫描线	刘汝佳